Quicksort

Part II Sorting and Order Statistics

Scope and prerequisites

Partitioning establishes relations between regions and a pivot; recursion is still needed to sort. Follow Lomuto's invariant through every numerical state, then distinguish worst-case time, expected comparisons over internal randomness, and stack depth, including duplicate-key degeneration. Prerequisites are array partitioning, recursion, and expectation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

- Implement partition with its invariant.
- Distinguish worst, best, and expected Quicksort costs.
- Analyze random pivots and adversarial input order.

Sections 7.1–7.4 cover the algorithm, performance, randomization, and analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.1 Description of quicksort

Choose a pivot, separate other keys by their relation to it, and recursively sort both sides. Partition alone does not order keys within a side. Use 1-based Lomuto partition with pivot at the final position r.

```
quicksort(A,p,r):
    if p>=r: return
    q=partition(A,p,r)
    quicksort(A,p,q-1)
    quicksort(A,q+1,r)
partition(A,p,r):
    pivot=A[r]; i=p-1
    for j=p..r-1:
        if A[j]<=pivot:
            i++; swap(A[i],A[j])
    swap(A[i+1],A[r])
    return i+1
```

Empty and singleton intervals are bases. Both child intervals exclude q and strictly shrink. Including q again can cause infinite recursion when the pivot reaches an endpoint.

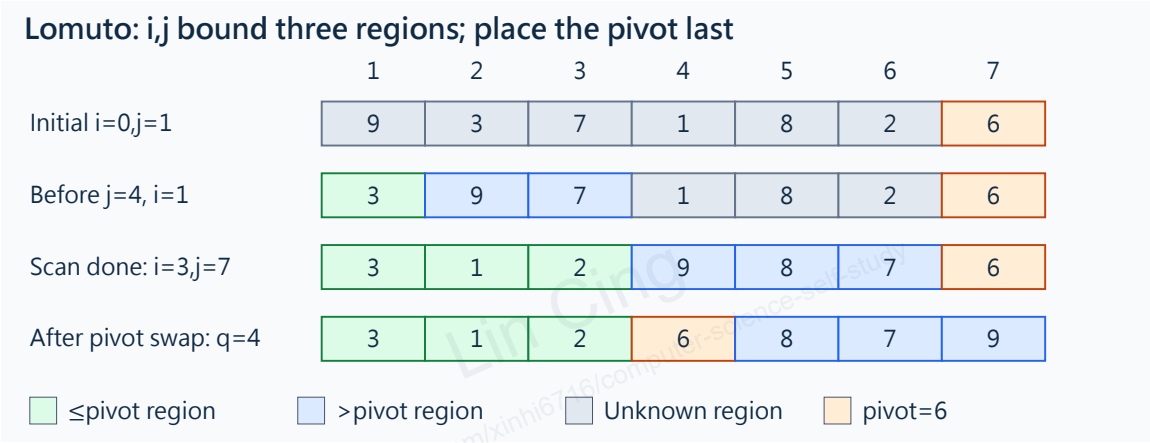
Lomuto's loop invariant

Before processing j, $A[p..i] \leq \text{pivot}$, $A[i+1..j-1] > \text{pivot}$, $A[j..r-1]$ is unknown, and $A[r] = \text{pivot}$. Empty intervals use the usual empty-set convention.

Initially $i=p-1, j=p$, so the first two regions are empty. If $A[j] \leq \text{pivot}$, increment i and swap. The small key joins the $\leq$ region; the displaced old value belonged to the $>$ region and remains there after j advances. If $A[j] > \text{pivot}$, advancing j expands only the $>$ region.

At $j=r$, no unknown keys remain. Exchange the pivot with $A[i+1]$, setting $q=i+1$. Left keys are $\leq \text{pivot}$ and right keys $> \text{pivot}$. Swaps preserve the multiset; both sides are smaller. Induction on interval length then proves that sorting each side and joining the pivot sorts the whole interval.

Lomuto's loop invariant (continued)



Complete partition trace

Let A=[9,3,7,1,8,2,6],p=1,r=7,pivot=6, initially i=0.

j	Test	New i	Array
1	9>6	0	[9,3,7,1,8,2,6]
2	3≤6	1	[3,9,7,1,8,2,6], swap 1,2
3	7>6	1	[3,9,7,1,8,2,6]
4	1≤6	2	[3,1,7,9,8,2,6], swap 2,4
5	8>6	2	[3,1,7,9,8,2,6]
6	2≤6	3	[3,1,2,9,8,7,6], swap 3,6
End	place pivot at i+1	q=4	[3,1,2,6,8,7,9], swap 4,7

Left [3,1,2] is ≤6 and right [8,7,9] is >6; neither is sorted internally. Partition performs exactly r−p comparisons, Θ(r−p+1) time, and O(1) extra space.

7.2 Performance of quicksort

An always-minimum or always-maximum pivot gives child lengths 0 and n−1, hence (n−1)+(n−2)+…+1=n(n−1)/2 comparisons. Sorted distinct input with a fixed final pivot is an entire worst-case family, not just a small observed example.

An upper bound must cover all splits. For worst comparison count C(n), child sizes a+b=n−1 and induction give

$$C(a)+C(b)+n-1 \le \frac{a(a-1)}{2}+\frac{b(b-1)}{2}+n-1=\frac{n(n-1)}{2}-ab\le\frac{n(n-1)}{2}.$$

Bases C(0)=C(1)=0 close the proof; endpoint splits attain equality. Each selected pivot disappears, so there are O(n) calls with constant fixed overhead. Total time O(n+C)=O(n^2), matching the family lower bound.

If the larger side is always at most ρn for fixed ρ<1, each level processes at most n keys and maximum depth is O(log_(1/ρ)n), giving O(n log n). Even a 1:9 split works.

7.2 Performance of quicksort (continued)

Perfectly balanced n=2^k−1 has k levels. Depth d processes n−(2^d−1) keys; during the first half of levels this is Θ(n), giving Ω(n log n) for that execution and matching the upper bound. This uses actual recursive work, rather than misapplying the comparison-sorting worst-input lower bound to every input.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Duplicate keys defeat this two-way partition

On [5,5,5,5,5], every $\leq$ test succeeds and q=r, leaving n−1 keys on the left. Randomly exchanging a pivot to the end still exchanges a 5, giving ten comparisons here and n(n−1)/2 generally.

The following O(n log n) random-pivot analysis therefore assumes distinct keys. For many duplicates, use <,=,> regions and never recurse on the equal region. Chapter 9 gives full interval boundaries and rank conversions.

Random pivots do not remove this duplicate-key premise. Stability is separate: long-distance swaps can change equal-key identity order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Recursion space and the smaller-side transformation

Two recursive calls can produce n stack levels. In-place partitioning does not make the whole algorithm unconditionally constant-space.

```
while p<r:
    q=partition(A,p,r)
    if q-p < r-q:
        quicksort-small-stack(A,p,q-1)
        p=q+1
    else:
        quicksort-small-stack(A,q+1,r)
        r=q-1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The actual recursive call has size at most (n−1)/2. The larger side continues in the current frame through a while loop. Thus sizes halve along a frame chain, giving worst-case O(log(n+1)) stack. Every partition still executes, and worst time remains $\Theta(n^2)$; reducing stack does not prove O(n log n) time.

7.3 A randomized version of quicksort

On current A[p..r], choose uniform k in p..r, swap A[k],A[r], then apply the same partition. Repeat uniform selection on every current child. With distinct keys, pivot rank is uniform in 1..n, so every fixed input order has the same expected analysis.

Expectation concerns internal random choices, not a uniform user-data assumption. An execution can keep selecting endpoints and cost $\Theta(n^2)$. The next proof establishes an expectation, not a per-execution or an unproved tail bound.

The ideal model requires uniform sampling conditional on current state. A fixed public pseudorandom sequence is a reproducible implementation choice whose adversarial guarantees need separate justification. A random API call by itself is not a proof of mathematical uniformity.

7.4 Analysis of quicksort

Write distinct sorted keys $z_1<\ldots<z_n$. For i<j, let X_{ij} indicate any direct comparison. A pair compares at most once: one endpoint is the pivot and is removed afterward.

Consider the rank interval $\{z_i,\ldots,z_j\}$. Until one of these keys becomes pivot, its endpoints remain in one subproblem. Outside pivots keep the entire interval in one child and cannot separate the endpoints.

If the first interval pivot is either endpoint, the two compare. If it is interior, they enter separate children and never compare. Each interval key is equally likely to be first, giving $P(X_{ij}=1)=2/(j-i+1)$. The event requires a first **endpoint** pivot, not just any first pivot in the interval.

Exact expected comparison sum

Let $C=\sum_{i<j}X_{ij}$. Linearity, without independence, gives

$$E[C]=\sum_{i<j}\frac{2}{j-i+1}=2\sum_{k=2}^{n}\frac{n+1-k}{k}=2(n+1)H_n-4n.$$

The harmonic bounds $\ln(n+1)\le H_n\le 1+\ln n$ give $\Theta(n\log n)$ expected comparisons. Loop work and recursive overhead are of the same order, giving $O(n\log n)$ expected time. These indicators are dependent, so variance or high-probability calculations cannot assume their independence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

All comparison probabilities for four ranks

Pair	Interval size	Probability
(1,2),(2,3),(3,4)	2	1 each
(1,3),(2,4)	3	2/3 each
(1,4)	4	1/2

The sum is $3+4/3+1/2=29/6$, matching $2\cdot 5\cdot H_4-16$. Endpoints 1,4 separate if pivot 2 or 3 comes first and compare if 1 or 4 comes first. Each first choice has probability 1/4.

Adjacent ranks always comparing does not imply every pair compares. Nor is 29/6 the cost of one execution; four keys can require six comparisons in the worst case. It is an average over internal random choices.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 7.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study